Практическая работа № 1

Модульное тестирование. Второй вариант. 2026 год.

Цель работы

Проверить расчёт времени в классе EstimatePolicy, написать отдельные тесты на допустимые и недопустимые значения и показать, что тесты обнаруживают ошибку в проверке верхней границы.

Правила

Допустимы целые значения от 1 до 8 включительно. Каждый балл соответствует 15 минутам. Для остальных значений метод выбрасывает IllegalArgumentException с сообщением points must be between 1 and 8. Ожидания определены из этих правил до запуска тестов.

Тестовые сценарии

Источник	Вход	Ожидание
Исходный пример	1	15 минут
Исходный пример	0	Исключение и сообщение
Разбор занятия	2	30 минут
Разбор занятия	9	Исключение
Второй вариант, задание 1	4	60 минут
Второй вариант, задание 2	7	105 минут
Второй вариант, задание 3	-2	Исключение
Второй вариант, задание 4	11	Исключение и сообщение

В первом задании второго варианта для 4 баллов указаны и 45, и 60 минут. В тесте принято 60, поскольку 4 × 15 = 60. Исходные примеры и тесты из разбора сохранены; всего 8 отдельных методов.

Основной класс

```
public int minutes(int points) {
    if (points < 1 || points > 8) {
        throw new IllegalArgumentException(
                "points must be between 1 and 8");
    }
    return points * 15;
}
```

Проверки и эксперимент

Расчёт времени

```
@Test
void fourPointsTakeSixtyMinutes() {
    EstimatePolicy policy = new EstimatePolicy();
    int actual = policy.minutes(4);
    assertEquals(60, actual);
}
```

Сначала создаётся объект, затем вызывается метод, после этого проверяется результат. Эти шаги соответствуют Arrange - Act - Assert. Эталон 60 взят из требования и не вычисляется через проверяемый метод. Для 7 баллов аналогично проверяется 105 минут.

Исключения

```
assertThrows(IllegalArgumentException.class,
        () -> policy.minutes(-2));

IllegalArgumentException error = assertThrows(
        IllegalArgumentException.class,
        () -> policy.minutes(11));
assertEquals("points must be between 1 and 8",
        error.getMessage());
```

Для -2 проверяется тип исключения. Для 11 дополнительно проверяется сообщение. Лямбда передаёт действие, которое JUnit вызывает внутри assertThrows; метод возвращает перехваченное исключение.

Задание 5: удаление верхней границы

Прогноз до запуска: если заменить if (points < 1 || points > 8) на if (points < 1), тест elevenPointsAreRejectedWithMessage упадёт. Значение 11 попадёт в расчёт и даст 165 вместо исключения. Исходный тест ninePointsAreRejected также должен упасть.

Состояние	Tests run	Failures	Errors	Skipped
Исходный код	8	0	0	0
Удалена верхняя граница	8	2	0	0
Обе границы восстановлены	8	0	0	0

Наблюдение совпало с прогнозом. Во втором запуске упали только проверки для 9 и 11 с сообщением:

```
Expected java.lang.IllegalArgumentException to be thrown,
but nothing was thrown.
```

После восстановления условия все 8 тестов прошли. Содержимое тестового класса во время трёх запусков оставалось неизменным; контрольная сумма записана в logs/summary.json.

Результат и запуск

Вывод

Проверены расчёты для 4 и 7 баллов, отклонение -2 и 11, а также исходные примеры занятия. При удалении верхней границы тесты обнаружили дефект; после восстановления обеих границ все 8 сценариев прошли. В финальной версии сохранена правильная реализация.

Как воспроизвести

Использовать JDK 25 или новее. Команды выполнять из папки 01. Maven Wrapper входит в проект; первая загрузка Maven и библиотек требует сети.

```
# macOS / Linux
sh ./mvnw test

# Только тестовый класс
sh ./mvnw "-Dtest=EstimatePolicyTest" test

# Windows PowerShell
.\mvnw.cmd test
```

Версии проекта: Java release 25, JUnit 6.1.3, Maven Wrapper 3.10.0, Compiler Plugin 3.15.0 и Surefire 3.6.0. Фактическая проверка проведена с JDK 26.

Материалы проверки

logs/run-success.log - исходный успешный запуск.
logs/run-failed.log - запуск с удалённой верхней границей.
logs/run-restored.log - успешный запуск после восстановления.
logs/run-*.xml - результаты каждого тестового метода.
logs/prediction.txt - прогноз до эксперимента.
logs/summary.json - числа тестов, коды завершения и контрольные суммы.

Журналы получены реальными запусками Maven в отдельной копии проекта. Пути к локальным папкам заменены на <project> и <home>. XML содержит результаты сценариев без свойств локального окружения.

Что проверяет эта работа

Это модульные тесты одного класса. Они проверяют расчёт и контракт недопустимого ввода. Интерфейс, база данных и сеть в этой работе не используются. Успех этих сценариев не доказывает отсутствие всех возможных ошибок.

Источники

PDF «Тестирование. Практика 1.pdf», 2026, страница 3.
Урок: https://lms.ctutp.ru/lms/courses/d7jsglqb09/learn/1-1
Стартовый проект: https://cloud.mail.ru/public/cMdk/S87xYy5WG